Ajaia Docs — System Architecture

Author: Sadat Shahriar Arko
Live Deployment: https://ajaia-assessment-iota.vercel.app
Technology Stack: Next.js 16, React 19, TypeScript, MongoDB Atlas, Tiptap, Mammoth.js, jsPDF, Vercel

1. System Architecture

Ajaia Docs follows a full-stack serverless architecture built with Next.js 16 using the App Router. The application is deployed on Vercel, where the React 19 frontend communicates with REST API route handlers through JSON and multipart/form-data requests.

MongoDB Atlas serves as the primary database, with Mongoose managing database operations. The MongoDB connection is maintained through a cached connection mechanism implemented in lib/mongodb.ts to reduce unnecessary reconnections across serverless invocations.

2. Data Models and Authentication

The application uses two primary MongoDB collections.

User Collection (models/User.ts)

Stores user information, including full name, normalized lowercase email address, and SHA-256 password hashes. Authentication logic is implemented in lib/auth.ts. Session information is maintained through HTTP-only cookies with the SameSite=Lax policy. Protected API endpoints verify the session on the server before processing requests.

Document Collection (models/Document.ts)

Stores document information, including title, rich-text HTML content, owner ID, owner name, owner email, and timestamps. It also contains a sharedWith array that stores collaborator email addresses and their assigned roles (editor or viewer). This structure allows documents to be shared with users before they register.

3. Role-Based Access Control

Ajaia Docs implements role-based access control (RBAC) to manage document permissions. The application checks the user's effective role when accessing protected document endpoints, including /api/documents/[id], /share, and /import.

The system supports three roles:

- Owner: Has full control over documents, including reading, editing, sharing, managing collaborator permissions, and deleting documents.

- Editor: Can read documents, make changes through the auto-save mechanism, and append imported content.
- Viewer: Has read-only access to documents. Any attempt to modify or delete a document through protected backend endpoints is rejected with HTTP 403 Forbidden.

4. Real-Time Synchronization and File Processing

Automatic Saving and Synchronization

The editor uses a debounced auto-save mechanism. Local changes trigger a PATCH request after 500 milliseconds of inactivity. A background polling mechanism checks for remote updates every 1.5 seconds.

To prevent remote updates from interrupting active typing, the application uses a 1200-millisecond local-edit guard window. This helps reduce conflicts when multiple collaborators work on the same document.

Document Import

The application supports importing multiple document formats:

- DOCX files are processed server-side using Mammoth.js and converted into semantic HTML.
- Markdown files are parsed into structured HTML blocks.
- TXT files are converted into HTML blocks.

Imported content can either be used to create a standalone document or appended to an existing document, depending on the selected operation.

PDF Export

PDF generation is handled entirely on the client side using jsPDF. The application traverses the rendered document DOM and generates paginated A4 PDF files directly in the browser, avoiding additional server-side processing for exports.